

A Labwork 4 Report

Luc Lacotte

0.1 Introduction

This labwork is about re-implementing the grayscale filter of the previous labwork using **2D blocks and grids** instead of 1D ones, and experimenting with different block sizes.

0.2 Using 2D blocks and grids

To use 2D blocks and grids, I first changed the kernel function. I added a second thread index, `tidy`, computed along the y axis in the same way as `tidx`:

```
tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
```

Then, because there is a new dimension, I added `tidy` to the indexing of the source and destination arrays. The image no longer needs to be flattened, it is passed with its original shape (`height, width, 3`).

```
@cuda.jit
def grayscaleGPU2D(src, dst):
    tidx = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    tidy = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    g = np.uint8((src[tidx, tidy, 0] + src[tidx, tidy, 1] + src[tidx, tidy,
2]) / 3)
    dst[tidx, tidy, 0] = dst[tidx, tidy, 1] = dst[tidx, tidy, 2] = g
```

The block size and grid size also become 2D tuples:

```
blockSize = (32, 32)
gridSize = (math.ceil(img.shape[0] / 32), math.ceil(img.shape[1] / 32))
grayscaledGPUimg = grayscaleGPU2D[gridSize, blockSize](devInput, devOutput)
```

0.3 Experimenting with different block sizes

I modified the script to try every square block size from 2×2 to 32×32 , running the kernel 10 times for each block size and taking the average execution time.

```
for i in range(2, 33):
    times = []
    for j in range(10):
        ...
        blockSize = (i, i)
        gridSize = (math.ceil(img.shape[0] / i), math.ceil(img.shape[1] / i))
        t1 = time.time()
        grayscaleGPU2D[gridSize, blockSize](devInput, devOutput)
        cuda.synchronize()
        t2 = time.time()
        times.append(t2 - t1)
    avgTimes.append(np.mean(times))
```

These are the results I got (I removed the block size 1×1 because it was too slow and messed up the scale of the graph):

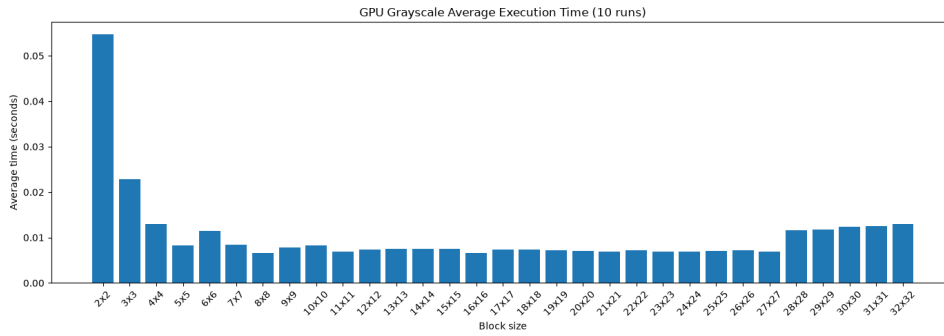


Figure 1: Average execution time of the 2D grayscale kernel depending on the block size (10 runs).

We can see that the smaller the block size, the slower the execution is. It stabilizes for block sizes between 5×5 and 27×27 , with 8×8 and 16×16 being slightly faster than the others. But from 28×28 to 32×32 , the execution time increases again.

This seems weird to me because it would mean that at 784 threads per block (28×28), some cores are not being utilized efficiently, while my GPU (RTX 3050 Ti Laptop) should be able to handle 1024 threads per block (32×32).

0.4 Comparison of 1D and 2D blocks and grids

To compare both approaches, I ran each kernel 10 times: the 1D version with a block size of 256 and the 2D version with a block size of 16×16 (also 256 threads per block).

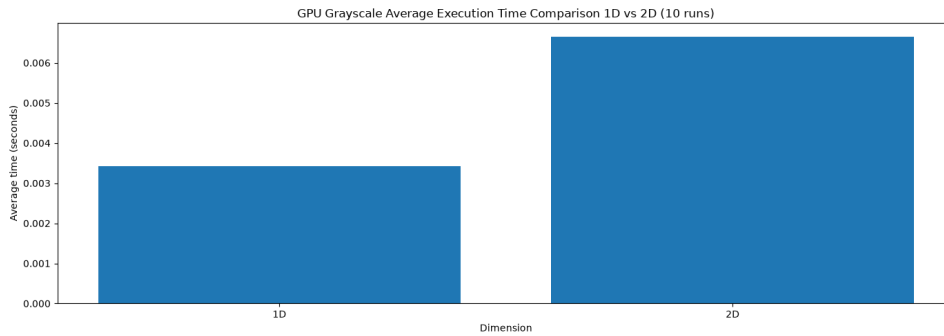


Figure 2: Average execution time of the 1D and 2D grayscale kernels (10 runs).

We can see that the 1D implementation is faster than the 2D implementation. Seems odd to me. But I don't know what is wrong.

0.5 Questions

1. It would be 16×16 , because we only have 512 threads per block, and 32×32 would be 1024 threads per block, which is too much for this GPU.
2. It would be 512 threads per block, because it allows us to use the maximum number of threads: $1536/3 = 512$, so this configuration perfectly fills the SM with 3 blocks of 512 threads.

The other options are not viable:

- 128 threads/block with at most 4 blocks would only give 512 threads;
 - 256 threads/block would only give 1024 threads;
 - 1024 threads/block would only allow a single block of 1024 threads.
3. Because we have 512 threads per block, the number of threads in the grid will be a multiple of 512, so in this case $4 \times 512 = 2048$ threads.